

Proposed Model

4.1. Introduction

The proposed model for skin disease detection is based on a CNN architecture designed to classify various types of skin diseases from dermoscopic images. The motivation behind choosing CNN is its proven effectiveness in image recognition tasks. We try to achieve more inspiring results and contribute to the development of skin disease classification applications using deep learning.

4.2. Dataset

Description of the Dataset:

The dataset used in this project is the ISIC 2019 dataset, a comprehensive collection of 25,352 dermoscopic images categorized into eight different skin lesion classes. The dataset is a resource for developing and evaluating skin diseases detection algorithms and conducting dermatological research. It was published on December 17, 2018, by a group of researchers including Tschandl, Codella and others.

Dataset Composition:

The dataset is divided into three subsets:

Training Set: 21,512 images (84.8%)

Validation Set: 1,910 images (7.5%)

Test Set: 1,930 images (7.6%)

The images in the dataset vary in size, predominantly 600×450 pixels and 1024×1024 pixels and are stored in JPG format. These images were sourced from a variety of global healthcare providers, clinics, and research centers specializing in dermatology.

Focused Skin Diseases:

For the scope of this project, we focus on four specific skin diseases:

Skin Disease	Training Images	Validation Images	Test Images
Actinic Keratosis	944	76	75
Dermatofibroma	420	22	43
Melanocytic Nevus	4428	931	423
Vascular Lesion	347	27	24

Table 3 Skin Diseases That Have Been Worked On

Data Preprocessing and Augmentation:

Given the unbalanced distribution across labels, data augmentation techniques are crucial to enhance the variability and robustness of the model. The following augmentation techniques were applied to the training dataset and validation dataset

Augmentation Technique	Parameter
Rescaling	1.0/255
Rotation	rotation_range=40
Width Shift	width_shift_range=0.2
Height Shift	height_shift_range=0.2
Shear	shear_range=0.2
Zoom	zoom_range=0.2
Horizontal Flip	horizontal_flip=True
Fill Mode	fill_mode=nearest
Feature-wise Standard Normalization	featurewise_std_normalization=True

Table 4 Data Augmentations

Additionally, the dull razor technique was employed for hair removal to enhance image clarity and focus on the lesion area. All images were resized to 224×224 pixels to maintain uniformity and compatibility with the model input requirements.

4.3. Model Architecture

The model architecture consists of a base DenseNet121 model with custom convolutional, pooling, dropout, and fully connected layers tailored for skin disease classification. Batch Normalization and dropout layers were used for regularization and improved training performance.

The base model is DenseNet121, loaded with pre-trained ImageNet weights. It includes convolutional blocks with skip connections, facilitating feature reuse and gradient flow.

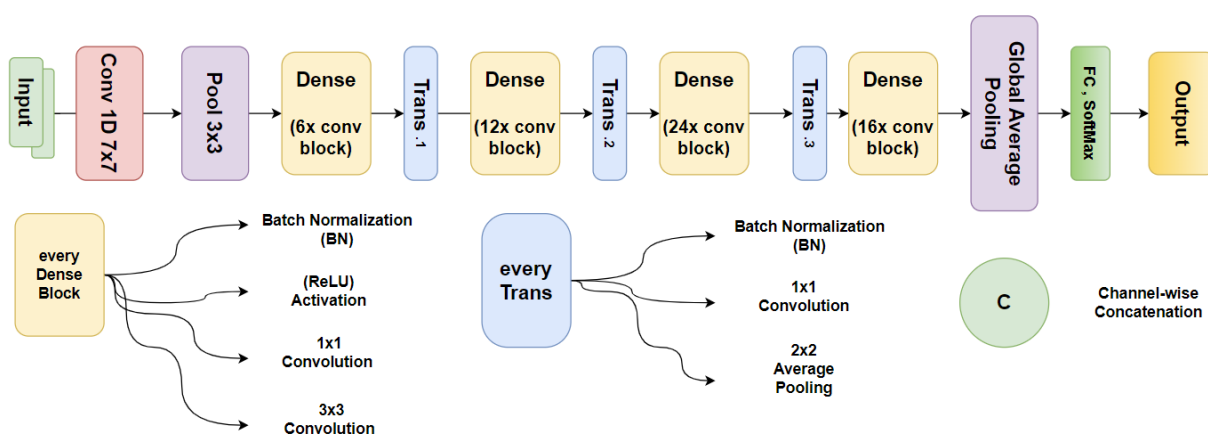


Fig11 DenseNet-121 architecture

Custom layers were added on top of the base model to adapt it to the specific task of skin disease classification.

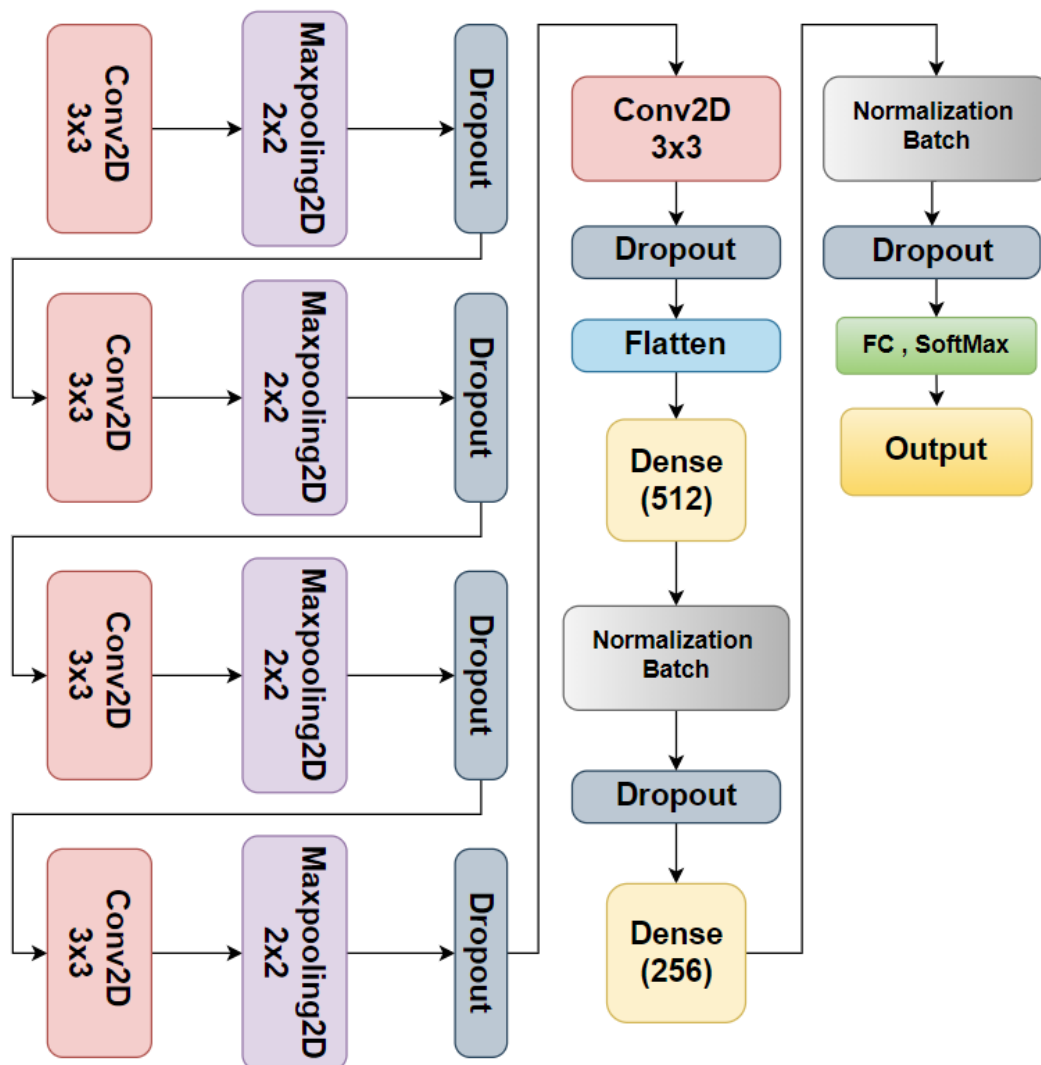


Fig12 Custom layers architecture

Convolutional Layers:

Layer 1: 256 filters, (3, 3) kernel size, ReLU activation, padding='same'

Layer 2: 128 filters, (3, 3) kernel size, ReLU activation, padding='same'

Layer 3: 64 filters, (3, 3) kernel size, ReLU activation, padding='same'

Layer 4: 32 filters, (3, 3) kernel size, ReLU activation, padding='same'

Pooling Layers:

Layer 1: Max pooling with (2, 2) pool size, padding='same'

Layer 2: Max pooling with (2, 2) pool size, padding='same'

Layer 3: Max pooling with (2, 2) pool size, padding='same'

Dropout Layers:

Dropout rate of 0.3 after Layer 1

Dropout rate of 0.2 after Layer 2 and Layer 3 and Layer 4

Flatten Layer:

Flatten the output of the convolutional layers to prepare for fully connected layers.

Fully Connected Layers:

Layer 1: 512 units, ReLU activation, with BatchNormalization and dropout rate of 0.5

Layer 2: 256 units, ReLU activation, with BatchNormalization and dropout rate of 0.3

Output Layer:

Dense layer with SoftMax activation for multi-class classification.

4.4. Training Procedure

The model was trained on a system with the following specifications:

RAM: 16 GB, GPU: NVIDIA GTX 1650 Ti, CPU: Intel Core i7-10750H,
Framework: TensorFlow 2.10, Python Version: 3.10, CUDA Toolkit: 11.2,
cudnn: 8.1.0, Package Manager: Anaconda 2.5.2

The hyperparameters used for training the model are as follows:

Epochs: 100, Batch Size: 100, Learning Rate: 0.001, Optimizer: Adam, Loss Function: Categorical Cross-Entropy

To evaluate the performance of the model, the following metrics were used:

Accuracy: Measures the overall correctness of the model's predictions.

Precision: Indicates the proportion of true positive results among all positive predictions.

Recall: Represents the proportion of true positive results among all actual positive cases.

F1-Score: The harmonic means of precision and recall, providing a balance between the two.

These metrics were chosen to provide a comprehensive evaluation of the model's performance, ensuring that both the accuracy and the quality of the predictions are considered.

Experimental Results

5.1. Main experiment

This experiment was carried out using densenet121, and Cubic SVM was combined with it, as mentioned in the proposed model chapter, and the evaluation matrix contains accuracy, precision, Recall, F1-Score

Accuracy measures the overall correctness of the model by comparing the number of correct predictions to the total number of predictions.

$$\text{Accuracy} = \frac{Tp + Tn}{Tp + Tn + Fp + Fn}$$

Precision measures the proportion of positive identifications that were actually correct.

$$\text{Precision} = \frac{Tp}{Tp + Fp}$$

Recall measures the proportion of actual positives that were correctly identified.

$$\text{Recall} = \frac{Tp}{Tp + Fn}$$

Since Precision and Recall can be defined as above, substituting them into the F1-Score equation gives:

$$\text{F1Score} = \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

The table below presents a comprehensive evaluation of our classification model's performance across four types of skin lesions: Actinic Keratosis, Dermatofibroma, Melanocytic Nevus, and Vascular Lesion. This table includes key metrics such as Precision, Recall, F1-Score, and Support, providing a detailed assessment of how well the model predicts each class.

	Precision	Recall	F1-Score	Support
Actinic keratosis	0.97	0.89	0.93	75
Dermatofibroma	0.85	0.95	0.9	43
Melanocytic nevus	0.99	0.98	0.98	423
Vascular lesion	0.8	1	0.89	24
Accuracy			0.96	565
Macro avg	0.9	0.96	0.93	565
Weighted avg	0.97	0.96	0.97	565

Table 5 Model Evaluation Metrics of densenet121

Overall, the model achieves an impressive accuracy of 0.96. The macro average and weighted average metrics also indicate strong and consistent performance across all classes, validating the robustness of our classification model. These metrics are crucial for understanding the strengths and potential areas for improvement in our model's predictions.

The integration of a **Cubic SVM** classifier with DenseNet121 further enhanced the accuracy, achieving a test accuracy of **0.99** in the best experimental setup.

The below image represents the confusion matrix for our classification model, which was trained to identify four different types of skin lesions: Actinic Keratosis, Dermatofibroma, Melanocytic Nevus, and Vascular Lesion. A confusion matrix is a useful tool for evaluating the performance of a classification algorithm. It provides a detailed breakdown of correct and incorrect predictions, allowing us to see how well the model distinguishes between different classes.

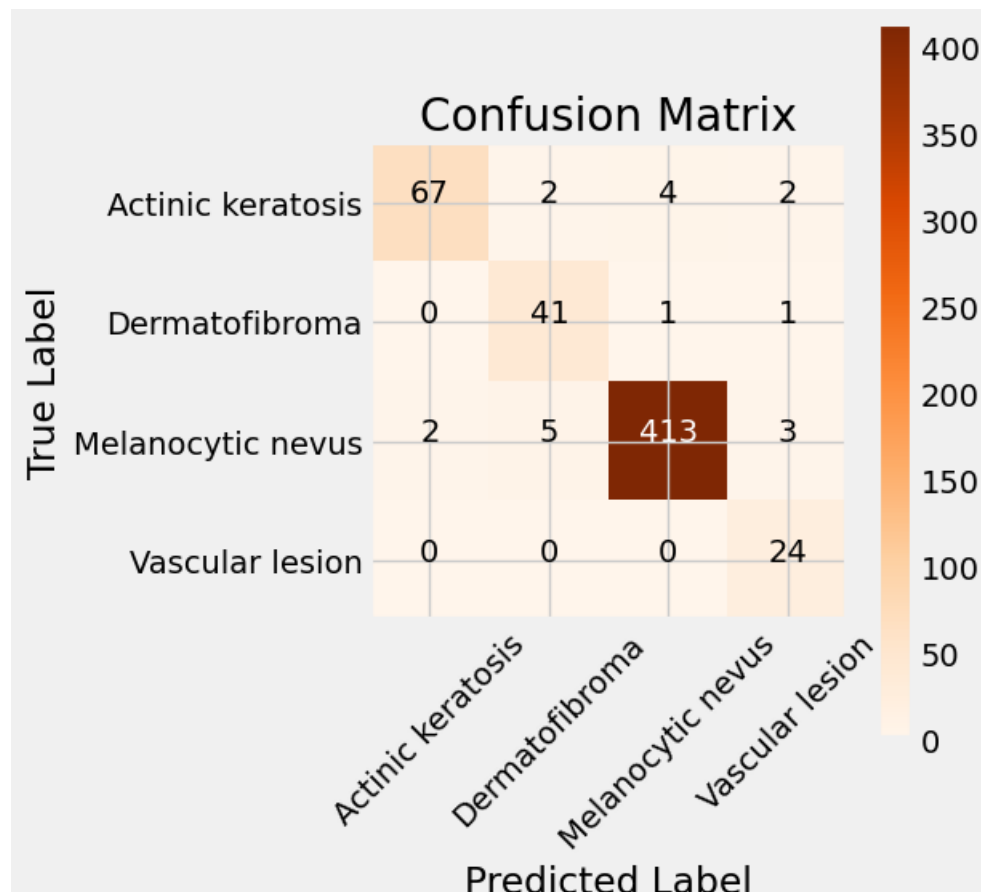


Fig 13 Confusion matrix of densenet121

5.2. Experiments comparisons

We have done many experiments where we adjust each hyperparameter independently just until we get the best result for the model containing that hyperparameter, and then we fixed the best result for that hyperparameter and change another hyperparameter until we get the best result and so on with all hyperparameters such as target size, unfreeze layers ,epochs, batch size, learning rate, dropout, early stopping, reduced learning rate, loss function, and optimizer. Therefore, we obtained very many results and experiments, so we presented a comparison between the best of these results in a way that shows the sequence in development.

Date	2-Apr	3-May	15-Apr using Cubic SVM method	30-Apr Cubic SVM method Under sampling
Epochs	10	15	100	100
Batch Size	32	64	100	100
Target Size	224,224	250,250	224,224	224,224
Reduce Learning Rate	No	Yes (factor=0.1, patience=15)	No	No
Unfreeze Layers	30 top layers	30 top layers	30 top layers	30 top layers
Early Stop	No	No	(monitor='loss', mode='min', patience=10)	(monitor='loss', mode='min', patience=10)
Dropout	0.3 0.5	0.5 0.5	0.5 0.3	0.5 0.3
Test Accuracy	0.86	0.914	0.959	0.964
Test Loss	0.717	0.422	0.137	0.127
Cubic SVM Test Accuracy	No	No	0.9	0.99
Cubic SVM Test Loss	No	No	0.09999	0.01

Table 6 Experiments Comparisons

Web development

6.1. Front-End Development

Our skin disease classification web application features a thoughtfully designed front-end that prioritizes user experience, information delivery, and responsiveness. This section provides an overview of the structure, technologies, libraries, and methodologies used in developing our front-end.

The website opens with a Header section that includes the website logo and a navigation bar with links to Home, Discover, App, and About Us sections. This layout ensures easy navigation and a cohesive user experience right from the start.

Following the header, the Intro Section engages users by introducing the application's purpose and value. This section aims to immediately capture the user's interest and provide a clear understanding of what the app offers and how it can benefit them.

Next, the Discover Dermatology section educates users about the various skin diseases our app can classify. It provides brief and informative descriptions of each condition, helping users gain a better understanding of their potential diagnoses and the importance of dermatological health.

The Consultation Section is designed to facilitate user interaction with dermatological experts. It provides a platform for users to seek more detailed information and professional advice about their skin conditions, enhancing the overall value of the application.

In the App Section, users can upload images of their skin conditions via an upload button or drag-and-drop functionality. Once an image is uploaded, the app processes it to classify the disease, offering a diagnosis along with treatment advice. This section is the core functionality of our application, providing users with immediate and actionable insights into their skin health.

The About Us section provides information about the development team, highlighting the expertise and efforts behind the project. It gives users a glimpse into the people responsible for the app, fostering trust and transparency.

Following this is the Acknowledgment Section, where we express our gratitude to the engineering and medical supervisors who contributed to the project. This section is a testament to the collaborative efforts that made the application possible.

Finally, the Footer mirrors the header with the navigation bar and logo, ensuring consistency and easy access to other sections of the website. It helps users navigate the site seamlessly and ensures that important links are always within reach.

The website is built using React v18, a powerful framework that allows us to create a dynamic and responsive user interface. This ensures that the website performs optimally across various devices, from desktops to mobile phones.

To enhance functionality, we employed several libraries. Axios is used for making API requests to the backend, efficiently handling responses and errors. React-Dropzone provides a seamless drag-and-drop interface for image uploads, enhancing the user experience in the App section. React-Slideshow is utilized to create engaging and interactive slideshows, adding a visually appealing element to the site.

In terms of tools, we used ESLint for code linting, which helps maintain high code quality and consistency by identifying and correcting issues in JavaScript code. Vite was employed for module management, significantly improving the development experience by providing a fast and efficient build tool.

Our data flow is designed to ensure smooth interaction between the front-end and backend. Using Axios, the front-end fetches data from the backend API, handling responses and errors to provide a seamless user experience. State

management is another critical aspect, where the application state is efficiently updated to reflect changes, ensuring that the user interface remains dynamic and responsive.

Overall, our front-end development strategy focuses on creating a robust, user-friendly, and responsive web application. By leveraging modern technologies, libraries, and tools, we have developed an interface that not only meets the functional requirements but also provides an engaging and informative user experience.

6.2. Back-End Development

Our back-end development is focused on providing a robust and efficient server that handles image classification requests for skin diseases. The back-end is developed using Flask, a lightweight web framework for Python, specifically version 3.0.3. This framework allows us to create web servers and APIs with minimal setup and code, making it an ideal choice for our application.

We start by importing the necessary libraries. Flask is used for creating the web server, `flask_cors` for handling Cross-Origin Resource Sharing (CORS) to allow our front-end to communicate with the back-end, and TensorFlow for loading and using our pre-trained machine learning model. Additionally, we use the Keras API, which is part of TensorFlow, for image preprocessing and model predictions.

The Flask application is initialized, and CORS is enabled to allow requests from different origins. This setup is crucial for enabling seamless interaction between our front-end (which might be hosted on a different domain) and the back-end.

We then load our pre-trained DenseNet model, which is stored in the 'densenet_svc_model.h5' file. This model is designed to classify skin diseases based on the images provided by the users. The model was trained on a dataset of skin disease images and can predict one of several classes: Actinic keratosis, Dermatofibroma, Melanocytic nevus, and Vascular lesion.

To provide a simple interface for users and ensure that our server is working, we define a home route (/) that returns a greeting message when accessed. This route serves as a basic check to confirm that the server is running correctly.

The core functionality of our back-end lies in the (/upload) route, which is set up to handle POST requests. When a user uploads an image through this endpoint, the server first checks if the file is present in the request. If not, it returns an error message. Once a file is detected, the image data is read and processed. The image is loaded and resized to 224x224 pixels to match the input size expected by our model. It is then converted to an array, expanded in dimensions, and normalized by dividing pixel values by 255.

The processed image is fed into the DenseNet121 model to make a prediction. The model outputs a set of probabilities corresponding to each class, and the class with the highest probability is selected as the predicted class. This class is then mapped to its corresponding label from the class_labels list.

Finally, the server responds with a JSON object containing the predicted label. This response is sent back to the front-end, which can then display the result to the user.

Overall, our back-end leverages Flask to create a responsive and efficient server that integrates a machine learning model for classifying skin diseases, handling user uploads, and returning predictions in real-time.

6.3. Deployment

In this section, we outline the deployment process for our skin disease classification web application. The deployment strategy ensures that our application is accessible, scalable, and performs efficiently.

For deploying our application, we utilized DigitalOcean, a cloud infrastructure provider renowned for its simplicity and scalability. DigitalOcean offers virtual servers known as Droplets, which are highly

configurable and can be tailored to meet specific needs. For our project, we chose DigitalOcean's Basic Droplets, which provided us with 1 vCPU, 1GB RAM, and 35GB of storage. These resources are sufficient for our initial deployment and allow for easy scaling as the application's user base grows. The Droplets provided a stable and robust environment for running our application, ensuring that it performs efficiently and remains responsive under load.

To host our React application and manage HTTP requests efficiently, we employed Nginx as our web server. Nginx is known for its high performance, stability, and low resource consumption, making it an ideal choice for serving our application. It handles incoming traffic, serves static files, and proxies API requests to the Flask back-end seamlessly.

The deployment process began with setting up a new Droplet on DigitalOcean, selecting the Basic Droplet configuration to meet our requirements. After accessing the Droplet via SSH, we installed the necessary software packages, including Nginx, Python, and required Python libraries. This initial setup provided the foundational environment required to run both our front-end and back-end applications.

The React application was built using the build process, generating static files that were then moved to the appropriate directory for Nginx to serve. Concurrently, the Flask back-end application, along with the machine learning model, was deployed and configured to run as a service. This ensures that the back-end remains active and responsive to incoming API requests, providing the necessary functionality for our application.

Configuring Nginx to serve the React application involved setting it up to handle static files and proxy API requests to the Flask back-end.

By leveraging DigitalOcean's scalable infrastructure and Nginx's efficient web serving capabilities, we have deployed a robust and scalable web application. This setup not only ensures high availability and performance but also provides a foundation for future growth and enhancements.

Conclusion and Future work

7.1. Conclusion

In conclusion, we explored key skin disease datasets, including HAM10000, ISIC 2019, PAD-UFES-20, BCN20000, Derm7pt, ISIC 2017, DermNet, PH2, and Dermofit, and provided valuable insights into their structures and characteristics, and examine their limitations and proposed solutions aim to enhance the quality and reliability of these datasets, laying a foundation for their optimal utilization in decision-making processes and applications dependent on accurate data. Building on our exploration, our project will rely on the ISIC 2019 dataset.

In our analysis of deep learning models such as ResNet-152, ResNet101, Inception-v3, DenseNet-121, DenseNet-201, MobileNet v1, MobileNet v2 & LSTM, VGGNet, and VGG16, we have gained a comprehensive understanding of their effectiveness in skin disease classification. By delving into their origins, dataset usage, preprocessing techniques, structures, layers, model-building processes, and result metrics, we are equipped with valuable knowledge for implementing robust deep learning model. Building on our exploration we chose DenseNet-121 algorithm based on their demonstrated capacity to capture intricate features and effectively manage complex image data.

The proposed CNN-based model for skin disease detection has demonstrated significant effectiveness in classifying various skin lesion types from dermatoscopic images. Leveraging the ISIC 2019 dataset, the model was trained and evaluated using a robust set of metrics, including accuracy, precision, recall, and F1-score, across four specific skin diseases: Actinic Keratosis, Dermatofibroma, Melanocytic Nevus, and Vascular Lesion.

Key highlights from our experimental results include:

- The model achieved a high overall accuracy of 0.96, indicating its strong capability in correctly classifying the skin lesion images.

- High precision and recall metrics across all classes, with particularly notable performance for Melanocytic Nevus (Precision: 0.99, Recall: 0.98, F1-Score: 0.98), underscoring the model's reliability in identifying the most common lesion in the dataset.
- Consistent macro and weighted average metrics validate the model's robustness and balanced performance across different classes, highlighting its potential for real-world dermatological applications.

To enhance the model's performance, extensive data preprocessing and augmentation techniques were employed to address the imbalance in the dataset and improve the model's generalization capabilities. The application of techniques such as rescaling, rotation, width and height shifts, shear, zoom, horizontal flip, and feature-wise standard normalization contributed to the model's high accuracy and robustness.

Furthermore, the model's architecture, based on DenseNet121 with custom convolutional, pooling, dropout, and fully connected layers, was carefully designed to optimize feature extraction and classification performance. The inclusion of pre-trained ImageNet weights and the strategic addition of batch normalization and dropout layers facilitated efficient training and regularization.

Experimental comparisons revealed that fine-tuning hyperparameters such as epochs, target size, learning rate, dropout rates, and early stopping criteria significantly impacted the model's performance. The integration of a Cubic SVM classifier further enhanced the accuracy, achieving a test accuracy of 0.99 in the best experimental setup.

In summary, the proposed model exhibits excellent potential for accurate skin disease classification, paving the way for advanced dermatological diagnostic tools. The thorough evaluation and iterative optimization underscore the model's capability and readiness for practical deployment in clinical settings. Future work will focus on expanding the dataset to include

more diverse skin lesions and exploring additional augmentation and optimization techniques to further improve model performance.

7.2. Future work

In the next phases of development for our skin disease classification web application, we aim to enhance the platform's capabilities and extend its reach to a broader audience. Here are some key areas we plan to focus on:

Expanding Service Reach: We aim to increase the number of dermatologists available on our platform and expand the service to new regions. By doing so, we can provide more comprehensive coverage and ensure that patients from diverse locations have access to expert dermatological care. This expansion will involve targeted outreach and partnerships with medical institutions to onboard qualified dermatologists.

User Profile Management: To improve user experience and ensure continuity of care, we plan to introduce user profiles. This feature will allow users to keep track of their personal data, medical history, and previous diagnoses. By maintaining detailed medical records, users and doctors can access a patient's history seamlessly, facilitating more accurate diagnoses and personalized treatment plans.

Categorize Users Groups: We will enhance our platform by creating distinct categories for users, specifically dividing membership into patients and doctors. This differentiation will enable us to tailor the user interface and functionalities to meet the specific needs of each group. For patients, this includes simplified appointment scheduling and access to their medical history, while for doctors, it includes tools for patient management and collaboration with peers.

Research and Publication: Given our achievement of an accuracy rate of up to 0.99 in skin disease classification, we will prepare a comprehensive research paper detailing our methodologies and findings. This paper will contribute to the academic and medical communities by presenting new results in the field of dermatological diagnostics. Publishing our research will

not only validate our work but also provide a foundation for future advancements and collaborations in medical AI.

By focusing on these areas, we aim to not only improve our current service but also contribute meaningfully to the field of dermatology through innovation and research. Our commitment to enhancing user experience, expanding access to care, and advancing medical knowledge underpins our future development goals.

References

- [1] Asiedu, K. B. Retrieved, WHO's inaugural global meeting on Neglected Tropical Diseases affecting the skin urges intensified efforts to address their impact, World Health Organization. (2023, March 31), <https://www.who.int/news/item/31-03-2023-who-first-global-meeting-on-skin-ntds-calls-for-greater-efforts-to-address-their-burden>
- [2] Tschandl, P., Rosendahl, C., & Kittler, H. (2018). The HAM10000 dataset, a large collection of multi-sources dermatoscopic images of common pigmented skin lesions, Scientific Data, (2018) <https://doi.org/10.1038/sdata.2018.161>
- [3] Skin Cancer MNIST: HAM10000, Kaggle, (2018, September 20). <https://www.kaggle.com/datasets/kmader/skin-cancer-mnist-ham10000>
- [4] International Skin Imaging Collaboration. (n.d.), ISIC, <https://www.isic-archive.com/>
- [5] Pacheco, A. G. C., Lima, G. R., Salomão, A. S., Krohling, B. A., Biral, I. P., De Angelo, G. G., Alves, F. C. R., Esgario, J. G. M., Simora, A. C., Castro, P. B. C., Rodrigues, F. B., Frasson, P., Krohling, R. A., Knidel, H., Santos, M. C. S., Santo, R. B. D. E., Macedo, T. L. S. G., Canuto, T. R. P., & De Barros, L. F. S. PAD-UFES-20: A skin lesion dataset composed of patient data and clinical images collected from smartphones. Data in Brief, 32, 106221,(2020), <https://doi.org/10.1016/j.dib.2020.106221>
- [6] Combalia, M. BCN20000: Dermoscopic lesions in the wild, arXiv.org, (2019, August 6), <https://arxiv.org/abs/1908.02288>
- [7] J. Kawahara, S. Daneshvar, G. Argenziano and G. Hamarneh, "Seven-Point Checklist and Skin Lesion Classification Using Multitask Multimodal Neural Nets," in *IEEE Journal of Biomedical and Health Informatics*, vol. 23, no. 2, pp. 538-546, March 2019, <https://doi.org/10.1109/JBHI.2018.2824327>.
- [8] Codella, N. C. F., Gutman, D., Celebi, M. E., Helba, B., Marchetti, M. A., Dusza, S. W., Kalloo, A., Liopyris, K., Mishra, N., Kittler, H., &

- Halpern, A. Skin Lesion Analysis Toward Melanoma Detection: A Challenge at the 2017 International Symposium on Biomedical Imaging (ISBI), Hosted by the International Skin Imaging Collaboration (ISIC), ArXiv:1710.05006 [Cs], (2018), <https://arxiv.org/abs/1710.05006>
- [9] Dr Ian Coulson. DermNet. (2021). <https://dermnetnz.org/>
- [10] Mendonca, T., Ferreira, P. M., Marques, J. S., Marcal, A. R. S., & Rozeira, J. PH2 - A dermoscopic image database for research and benchmarking. 2013 35th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC), (2013), <https://doi.org/10.1109/embc.2013.6610779>
- [11] ADDI - Automatic computer-based Diagnosis system for Dermoscopy Images, (2013), <https://www.fc.up.pt/addi/ph2%20database.html>
- [12] Rees, Aldridge, Fisher, Ballerini, A Color and Texture Based Hierarchical K-NN Approach to the Classification of Non-melanoma Skin Lesions, Color Medical Image Analysis, Lecture Notes in Computational Vision and Biomechanics 6 (M. E. Celebi, G. Schaefer (eds.)), (2013) <https://licensing.edinburgh-innovations.ed.ac.uk/product/dermofitimage-library>
- [13] Mendes, D. B. Skin lesions classification using convolutional neural networks in clinical images. arXiv.org, (2018, December 6), <https://arxiv.org/abs/1812.02316>
- [14] Demir, A., Yilmaz, F., & Kose, O. Early detection of skin cancer using deep learning architectures: resnet-101 and inception-v3, TIPTEKNO 2019, <https://doi.org/10.1109/tiptekno47231.2019.8972045>
- [15] Al-Saedi, D. K. A., & Savaş, S. Classification of Skin Cancer with Deep Transfer Learning Method, Computer Science, (2022), <https://doi.org/10.53070/bbd.1172782>
- [16] Rehman, M. Z. U., Ahmed, F., Alsuhibany, S. A., Jamal, S. S., Ali, M. Z., & Ahmad, J. Classification of skin cancer lesions using explainable deep learning, Sensors, 22(18), 6915, (2022), <https://doi.org/10.3390/s22186915>